

מטלה 2 – מערכות הפעלההארה :

לפני ההגשה שמנו לב שיש לנו טעות נגררת קטנה בכל השלבים
תיקנו אותה לפני שהגשנו אבל קובץ הזה נשאר כמו שהוא , שכן לא ביקשתם קובץ הסברים צירפנו אותו למען
הנוחות .

למרות זאת הקובץ הזה עוזר בדוגמאות ההרצה וכו . בתיקון התייחסנו להדפסת שגיאות באופן תקין כך
שההרצה וכיסוי קוד נשארים כפי שהם .

מחסן אטומים

במטלה זו נתעסק באחסון אטומים של פחמן, חמצן, ומימן .

- אפשר לאחסן סדר גודל של 10 בחזקת 18 אטומים מכל סוג .

יש שימוש בפונקציית select

שלב ראשון:

נכתוב שרת בשם atom_warehouse המקבל התחברויות ב-TCP . לאחר שהתקבלה התחברות נשתמש
ב IOMUX על מנת לטפל בלקוחות מחוברים וכן לקבל התחברויות נוספות .

- השרת יקבל בתור ארגומנט ראשון ויחיד את הפורט עליו הוא מאזין

בנוסף נכתוב לקוח atom_supplier שמתחבר לשרת ושולח בקשות על פי ממשק משתמש .

הלקוח יקבל בתור ארגומנט ראשון את שם השרת , ובתור ארגומנט שני את הפורט אליו הוא התחבר .

צד השרת

- atom_warehouse

זהו שרת TCP **מרובה לקוחות** שמאזין לפקודות ADD (להוסיף אטומים) מלקוחות, שומר את הכמויות בזיכרון,
ומחזיר לכל לקוח את מספר האטומים הקיימים מסוג זה.

```
11 #define MAX_CLIENTS FD_SETSIZE // Default size at which select() can track maximum number of open files.
12 #define BUFFER_SIZE 1024
13
14 volatile sig_atomic_t shutdown_requested = 0; //When a SIGINT signal arrives (for example, pressing Ctrl+C), the server will know to terminate execution.
15
```

הגדרנו את המספר המקסימלי של הלקוחות כך FD_SETSIZE הוא גודל ברירת מחדל שבו select() יכול
לעקוב אחר **מספר מקסימלי של מספר חיבורים פעילים לשרת** . לכן MAX_CLIENTS, קובע כמה לקוחות
TCP יכולים להיות מחוברים בו-זמנית, מבלי שנחרוג ממה ש-select() מסוגל לטפל בו.

הגדרנו את זה :

```
int main(int argc, char* argv[]) {
    signal(SIGPIPE, SIG_IGN);
    signal(SIGINT, handle_shutdown);
```

signal(SIGPIPE, SIG_IGN)

אות SIGPIPE נשלח לתהליך כאשר הוא מנסה לכתוב write או send ל-socket שהצד השני שלו (הלקוח)
כבר סגר. באמצעות השורה הזו, אנו מורים למערכת **להתעלם מהאות**, (SIG_IGN = ignore), וכך ניתן לטפל
בשגיאה ידנית באמצעות בדיקת ערכי החזרה (errno) במקום שהשרת יקרוס.

signal(SIGINT, handle_shutdown);

אות SIGINT נשלח לתהליך כאשר המשתמש לוחץ על **Ctrl+C**.

ברירת המחדל היא סיום מידי של התהליך. כאן אנו מגדירים פונקציית טיפול בשם `handle_shutdown`, שתפקידה לעדכן משתנה גלובלי (`shutdown_requested = 1`)

צד הלקוח
atom-supplier

אנחנו בעצם מגדירים לו 3 מצבים

1. הרצת הלקוח באופן חוקי כדרישות המטלה 2 ארגומנטים שם שרת ומספר הפורט שאליו השרת התחבר לדוגמא

`./atom_supplier 127.0.0.1 4444`

2. במידה שהלקוח ירצה להתנתק אז הוא ירשום EXIT ויותר לא יהיה מחובר לשרת

3. הלקוח יכול לרשום SHUTDOWN ולגרום לשרת להתנתק לגמרי

בקידוד של הלקוח
אנחנו יוצרים חיבור TCP עם השרת

יש לנו שימוש בפונקציה כמו `getaddrinfo`

```
int status = getaddrinfo(server_ip, port_str, &hints, &res);
if (status != 0) {
    fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(status));
    return 1;
}
```

כלומר אנו מתרגמים כתובת ה-IP והפורט לכתובת רשת (`sockaddr`) שמוכנה לשימוש ב-`connect()`

לאחר מכן אנו בודקים שהפקודה היא מהצורות הבאות בלבד כולל שם האטום

ADD CARBON 4

ADD OXYGEN 4

ADD HDROGEN 4

אם כן: שולח לשרת, ומחכה ל-`unsigned int` שמחזיר את סך האטומים מאותו סוג.

דוגמת הרצה
ראשית נפעיל את השרת:

```
ron@Ron:~/OS_EX2/EX1$ ./atom_warehouse 4444
Server is listening on port 4444
```

בטרמינל אחר נפעיל את הלקוח

```
ron@Ron:~/OS_EX2/EX1$ ./atom_supplier 127.0.0.1 4444
Connected to server at 127.0.0.1:4444
Enter command ADD CARBON , OXYGEN , HYDROGEN: 
```

נשלח בקשות אל השרת

```
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD CARBON 4
Server returned count: CARBON = 4
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD OXYGEN 4
Server returned count: OXYGEN = 4
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD HYDROGEN 4
Server returned count: HYDROGEN = 4
Enter command ADD CARBON , OXYGEN , HYDROGEN: 
```

בצד השרת :

```
ron@Ron:~/OS_EX2/EX1$ ./atom_warehouse 4444
Server is listening on port 4444
New client connected: 127.0.0.1
Atoms warehouse: H=4, O=0, C=0
Atoms warehouse: H=4, O=4, C=0
Atoms warehouse: H=4, O=4, C=4

```

בצד הלקוח הוא כותב EXIT

```
Enter command ADD CARBON , OXYGEN , HYDROGEN: EXIT
send EXIT to server- client requests to disconnect
ron@Ron:~/OS_EX2/EX1$ 
```

ובצד השרת

```
Client requested EXIT.
```

בצד הלקוח עשינו חיבור של לקוח חדש

```
ron@Ron:~/OS_EX2/EX1$ ./atom_supplier 127.0.0.1 4443
Connected to server at 127.0.0.1:4443
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD CARBON 4
Server returned count: CARBON = 8
Enter command ADD CARBON , OXYGEN , HYDROGEN: 
```

בצד השרת :

```
Client requested EXIT.
New client connected: 127.0.0.1
Atoms warehouse: H=8, O=0, C=0

```

כעת נראה מה קורה שעושים SHUTDOWN

בצד הלקוח:

```
Server returned count: CARBON = 8
Enter command ADD CARBON , OXYGEN , HYDROGEN: SHUTDOWN
Sent SHUTDOWN. Closing client.
```

ובצד השרת

```
Shutdown
Server shut down.....
```

Code coverage -

הרצנו script על מנת שהטסטים שלנו יהיו אוטומטיים

קראנו לו testcase

הוא בודק את כל המקרים הבאים

- הרצת השרת ללא פורט
- הרצת הלקוח ללא port וללא host
- בפקודה ADD ההכנסה של מספר 0 או מספר שלילי
- הכנסת פקודה ADD באופן תקני של האטומיים החוקיים
- הכנסת פקודה ADD עם אטום שלא חלק משלושת האטומים
- הלקוח עושה EXIT
- הלקוח עושה SHUTDOWN
- לחיצה על ctrl c

התוצאות הן

```
Generating coverage reports...
File 'atom_warehouse.c'
Lines executed:91.59% of 107
Creating 'atom_warehouse.c.gcov'

Lines executed:91.59% of 107
File 'atom_supplier.c'
Lines executed:78.79% of 66
Creating 'atom_supplier.c.gcov'

Lines executed:78.79% of 66
```

זוהי תקין ולא 100% משום שהבדיקות של ההצלחה יצירת סוקט וכדומה לא נבדקות לדוגמא:

- בקובץ atom_warehouse.c.gcov

```
44      3: 40: int server_fd = socket(AF_INET, SOCK_STREAM, 0);
45      3: 41: if (server_fd < 0) {
46      #####: 42:     perror("socket failed");
47      #####: 43:     exit(EXIT_FAILURE);
```

```
61      2: 57:
62      2: 58:     if (listen(server_fd, 5) < 0) {
63      #####: 59:         perror("listen failed");
64      #####: 60:         close(server_fd);
65      #####: 61:         exit(EXIT_FAILURE);
66      -: 62:     }
```

```
21: 82: int activity = select(max_fd + 1, &readfds, NULL, NULL, NULL);
21: 83: if (activity < 0 && !shutdown_requested) {
#####: 84:     perror("select error");
#####: 85:     break;
-: 86: }
-: 87:
21: 88: if (FD_ISSET(server_fd, &readfds)) {
-: 89:     struct sockaddr_in client_addr;
7: 90:     socklen_t client_len = sizeof(client_addr);
7: 91:     int new_socket = accept(server_fd, (struct sockaddr*)&client_addr, &client_len);
7*: 92:     if (new_socket < 0) {
#####: 93:         perror("accept failed");
#####: 94:         continue;
-: 95:     }
```

- atom_supplier.c.gcov

```

#####: 31:      fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(status));
#####: 32:      return 1;
-: 33:  }
-: 34:
8: 35:  int sock = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
8: 36:  if (sock < 0)
-: 37:  {
#####: 38:      perror("socket failed");
#####: 39:      freeaddrinfo(res);
#####: 40:      return 1;
-: 41:  }
-: 42:
8: 43:  if (connect(sock, res->ai_addr, res->ai_addrlen) < 0)
-: 44:  {
#####: 45:      perror("Connection failed");
#####: 46:      freeaddrinfo(res);
#####: 47:      close(sock);
#####: 48:      return 1;
-: 49:  }

#####: 94:      perror("Send failed");
1*: 95:      break;
-: 96:  }
-: 97:
9: 98:      unsigned int response = 0;
9: 99:      ssize_t bytes_received = recv(sock, &response, sizeof(response), 0);
9: 100:      if (bytes_received != sizeof(response))
-: 101:      {
1: 102:          fprintf(stderr, "Failed to receive proper response from server\n");
1: 103:          break;
-: 104:      }
-: 105:
8: 106:      printf("Server returned count: %s = %u\n", atom_type, response);
-: 107:  }
-: 108:  else
-: 109:  {
9: 110:      ssize_t bytes_sent = send(sock, buffer, strlen(buffer), 0);
9: 111:      if (bytes_sent < 0)
-: 112:      {
#####: 113:          perror("Send failed");
#####: 114:          break;

```

```

#####: 121:      fprintf(stderr, "Failed to receive proper response from server\n");
#####: 122:      break;
-: 123:  }
-: 124:

```

שלב שני :

נרצה לשנות את המחסן , כלומר כעת המחסן יכול לשנות את מספר האוטומטים לפי המולקולות שהמחסן יכול לספק .
 המחסן שלנו יכול לספק מולקולות של מים (H_2O) , פחמן דו חמצני (CO_2) , גלוקוז ($C_6H_{12}O_6$) , אתנול (C_2H_6O) .

בדרישות השאלה השרת כרגע צריך לקבל בתור ארגומנט שני את הפורט עליו הוא יקבל בקשות בudp לדוגמה נתחייב להריץ כך
`./supplier_molecule 4444 5555`
 כך ש 4444 זה הפורט שהשרת יאזין בtcp ו 5555 זה הפורט שהשרת יאזין בudp

הכל נשאר כמו שלב ראשון רק שאנחנו מוסיפים את היכולת של השרת לבצע חיבור של UDP כלומר atom_warehouse משלב 1 עם שיפור של חיבור UDP ועדכון כמה אטומים נשאר במחסן אחרי שליחה של המולקולות

כעת נדבר על requester_molecule שהוא מבצע בקשות לשרת לצורך **אספקת מולקולות** לפי הפקודה DELIVER כלומר הוא מייצג לקוח UDP אשר שולח בקשות לשרת עבור יצירת מולקולות כגון DELIVER WATER 2 הכתובת והפורט של השרת מתקבלים כארגומנטים בשורת הפקודה.
 בכל הרצה, המשתמש מזין פקודת DELIVER, והלקוח שולח אותה דרך `sendto()` ליצור חיבור קבוע כיוון שמדובר ב-UDP
 לאחר מכן, מתבצעת קריאה ל-`recvfrom()` שמחזירה את תשובת השרת DELIVERED — או .CANNOT_DELIVER

כעת נדבר על

שלב ראשון : להפעיל את האזנה של השרת

```
ron@Ron:~/OS_EX2/EX2$ ./molecule_supplier 14444 15555
Server is listening on TCP 14444 and UDP 15555
```

שלב שני : למלא את מחסן האטומים דרך חיבור הTCP

```
ron@Ron:~/OS_EX2/EX2$ ./atom_supplier 127.0.0.1 14444
Connected to server at 127.0.0.1:14444
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD CARBON 4
Server returned count: CARBON = 4
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD OXYGEN 4
Server returned count: OXYGEN = 4
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD HYDROGEN 4
Server returned count: HYDROGEN = 4
Enter command ADD CARBON , OXYGEN , HYDROGEN: 
```

שלב שלישי : לשלוח בקשה בUDP

```
ron@Ron:~/OS_EX2/EX2$ ./molecule_requester 127.0.0.1 15555
Enter DELIVER commands (e.g., DELIVER WATER 2), or type EXIT to quit:
> DELIVER WATER 1
Server response: DELIVERED
> 
```

אחרי הבקשה נראה בשרת :

```
UDP DELIVER: WATER (1 units)
Atoms warehouse after delivery: H=2, O=3, C=4
```

- Code coverage

בדקנו אוטומטית בעזרת סקריפט את כל המקרים

- הכנת פקודות תקינות של DELIVER
- בדיקת המקרה בו אנחנו דורשים מולקולה אך המספר חסר
- בדיקת המקרה בו דורשים מולקולה שלא נמצאת בסעיף
- הרצת הלקוחות ללא ארגומנטים שהם צריכים לקבל
- הרצת השרת ללא הארגומנטים שהוא צריך לקבל
- כמו בשלב 2 – בדיקת התקינות של הפקודה ADD
- בדיקת קליטים לא טובים לשרת
- שליחת פקודת SHUTDOWN לשרת TCP וסגירתו.
- שליחת EXIT
- ניסיון שליחת EXIT גם דרך molecule_client
- לקוח אחד נשאר מחובר (ומבצע EXIT לאחר שהייה) ובו זמנית לקוח אחר שולח SHUTDOWN ומפיל את השרת.

```
Generating coverage reports...
File 'molecule_supplier.c'
Lines executed:89.77% of 176
Creating 'molecule_supplier.c.gcov'

Lines executed:89.77% of 176
File 'atom_supplier.c'
Lines executed:80.30% of 66
Creating 'atom_supplier.c.gcov'

Lines executed:80.30% of 66
File 'molecule_requester.c'
Lines executed:70.59% of 34
Creating 'molecule_requester.c.gcov'
```

נפרט את התוצאות ונסביר מדוע זה עדין תוצאות טובות למרות שהן לא מגיעות ל100%

- Molecule_supplier

```
#####: 43:      perror("TCP socket failed");
#####: 44:      exit(EXIT_FAILURE);
-: 45:  }
```

```

#####: 62:         perror("TCP listen failed");
#####: 63:         close(server_fd);
#####: 64:         exit(EXIT_FAILURE);
-: 65:     }
-: 66:
-: 67:     // UDP socket
3: 68:     int udp_fd = socket(AF_INET, SOCK_DGRAM, 0);
3: 69:     if (udp_fd < 0)
-: 70:     {
#####: 71:         perror("UDP socket failed");
#####: 72:         close(server_fd);
#####: 73:         exit(EXIT_FAILURE);
-: 74:     }

```

```

#####: 84:         perror("UDP bind failed");
#####: 85:         close(server_fd);
#####: 86:         close(udp_fd);
#####: 87:         exit(EXIT_FAILURE);
-: 88:     }

```

```

-: 114:     {
#####: 115:         perror("select error");
#####: 116:         break;
-: 117:     }
-: 118:
42: 119:     if (activity < 0)
-: 120:     {
1: 121:         if (errno == EINTR && shutdown_requested)
11: 122:             continue; // תעבור ללולאה ותבדוק את המשתנה
#####: 123:         perror("select error");
#####: 124:         break;
-: 125: }
-: 126:     // Handle new TCP connection
41: 127:     if (FD_ISSET(server_fd, &readfds))
-: 128:     {
-: 129:         struct sockaddr_in client_addr;
7: 130:         socklen_t client_len = sizeof(client_addr);
7: 131:         int new_socket = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
7*: 132:         if (new_socket < 0)
-: 133:         {
#####: 134:             perror("accept failed");
#####: 135:             continue;
-: 136:         }

```

כל אלו הן בדיקות עבור חוסר הצלחה, לדוגמא של יצירת סוקט.

Atom_supplier -

```

#####: 31:         fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(status));
#####: 32:         return 1;
-: 33:     }
-: 34:
11: 35:     int sock = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
11: 36:     if (sock < 0)
-: 37:     {
#####: 38:         perror("socket failed");
#####: 39:         freeaddrinfo(res);
#####: 40:         return 1;
-: 41:     }

```



```

-: 93:         {
#####: 94:             perror("Send failed");
#####: 95:             break;
-: 96:         }
-: 97:
9: 98:         unsigned int response = 0;
9: 99:         ssize_t bytes_received = recv(sock, &response, sizeof(response), 0);
9: 100:         if (bytes_received != sizeof(response))
-: 101:         {
#####: 102:             fprintf(stderr, "Failed to receive proper response from server\n");
#####: 103:             break;
-: 104:         }
-: 105:
9: 106:         printf("Server returned count: %s = %u\n", atom_type, response);
-: 107:     }
-: 108:     else
-: 109:     {
3: 110:         ssize_t bytes_sent = send(sock, buffer, strlen(buffer), 0);
3: 111:         if (bytes_sent < 0)
-: 112:         {
#####: 113:             perror("Send failed");
#####: 114:             break;
-: 115:         }
3: 116:         unsigned int response = 0;
3: 117:         ssize_t bytes_received = recv(sock, &response, sizeof(response), 0);
3: 118:         if (bytes_received != sizeof(response))
-: 119:         {
#####: 120:             fprintf(stderr, "Failed to receive proper response from server\n");
#####: 121:             break;
-: 122:         }

```

זה גם תקין כי אלו מקרים של בדיקות , לדוגמא במידה שהsend נכשל

- molecule_requester

```

#####: 11:         fprintf(stderr, "Usage: %s <server_ip> <port>\n", argv[0]);
#####: 12:         exit(EXIT_FAILURE);
-: 13:     }
-: 14:
6: 15:     const char *server_ip = argv[1];
6: 16:     int port = atoi(argv[2]);
-: 17:
-: 18:     // יצירת socket UDP
6: 19:     int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
6: 20:     if (sockfd < 0) {
#####: 21:         perror("socket failed");
#####: 22:         exit(EXIT_FAILURE);
-: 23:     }
-: 24:
-: 25:     struct sockaddr_in server_addr;
6: 26:     memset(&server_addr, 0, sizeof(server_addr));
6: 27:     server_addr.sin_family = AF_INET;
6: 28:     server_addr.sin_port = htons(port);
6: 29:     if (inet_pton(AF_INET, server_ip, &server_addr.sin_addr) <= 0) {
#####: 30:         perror("invalid address");
#####: 31:         exit(EXIT_FAILURE);
-: 32:     }

```

זה גם תקין מאותה סיבה

שלב שלישי :

השארנו את atom_supplier בדיוק אותו הדבר מהשלבים הקודמים, ולקחנו משלב שני את molecule_requester

ושידרגנו את השרת לdrinks_bar כך שבתוספת ליכולות של שלבים 1 ו-2 כעת נוסיף את היכולת של השרת לקבל דרך המקלדת את הפקודות הבאות

GEN SOFT DRINK

GEN VODKA

GEN CHAMPAGNE

הוספנו את החלק הבא:

```

249
250 if (FD_ISSET(STDIN_FILENO, &readfds))
251 {
252     char input[BUFFER_SIZE];
253     if (fgets(input, sizeof(input), stdin))
254     {
255         input[strcspn(input, "\n")] = 0;
256         char command[32], drink[32];
257         if (sscanf(input, "%31s %31s", command, drink) == 2 && strcmp(command, "GEN") == 0)
258         {
259             int can_make = 0;
260             if (strcmp(drink, "SOFT") == 0 || strcmp(drink, "SOFTDRINK") == 0 || strcmp(drink, "SOFT_DRINK") == 0)
261             {
262                 can_make = min3(hydrogen / 14, oxygen / 9, carbon / 7);
263                 printf("Can make %d SOFT DRINK(s)\n", can_make);
264             }
265             else if (strcmp(drink, "VODKA") == 0)
266             {
267                 can_make = min3(hydrogen / 20, oxygen / 8, carbon / 8);
268                 printf("Can make %d VODKA(s)\n", can_make);
269             }
270             else if (strcmp(drink, "CHAMPAGNE") == 0)
271             {
272                 can_make = min3(hydrogen / 8, oxygen / 4, carbon / 3);
273                 printf("Can make %d CHAMPAGNE(s)\n", can_make);
274             }
275             else
276             {
277                 printf("Unknown drink type.\n");
278             }
279         }
280         else
281         {
282             printf("Invalid GEN command.\n");
283         }
284     }
285 }

```

על מנת שנוכל לקבל כפקודות טקסט קלט מהמשתמש

עבור כל משקה הגדרנו כמה משקאות מסוג מסוים אפשר להכין לפי האטומים שיש במחסן כרגע (hydrogen, oxygen, carbon), מבלי לשנות את המלאי בפועל.

- דוגמת הרצה

```

ron@Ron:~/OS_EX2/EX3$ ./drinks_bar 4444 5555
Server is listening on TCP 4444 and UDP 5555

```

```

ron@Ron:~/OS_EX2/EX3$ ./atom_supplier 127.0.0.1 4444
Connected to server at 127.0.0.1:4444
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD CARBON 100
Server returned count: CARBON = 100
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD OXYGEN 100
Server returned count: OXYGEN = 100
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD HYDROGEN 100
Server returned count: HYDROGEN = 100
Enter command ADD CARBON , OXYGEN , HYDROGEN:

```

```

ron@Ron:~/OS_EX2/EX3$ ./drinks_bar 4444 5555
Server is listening on TCP 4444 and UDP 5555
New client connected: 127.0.0.1
Atoms warehouse: H=0, O=0, C=100
Atoms warehouse: H=0, O=100, C=100
Atoms warehouse: H=100, O=100, C=100
GEN SOFT DRINK
Can make 7 SOFT DRINK(s)
GEN VODKA
Can make 5 VODKA(s)
GEN CHAMPAGNE
Can make 12 CHAMPAGNE(s)

```

Code coverage -

גם בשלב זה עשינו סקריפט שיריץ את הטסטים באופן אוטומטי

וננתח את התוצאות

```

Generating coverage reports...
File 'drinks_bar.c'
Lines executed:90.77% of 195
Creating 'drinks_bar.c.gcov'

Lines executed:90.77% of 195
File 'atom_supplier.c'
Lines executed:80.30% of 66
Creating 'atom_supplier.c.gcov'

Lines executed:80.30% of 66
File 'molecule_requester.c'
Lines executed:76.47% of 34
Creating 'molecule_requester.c.gcov'

```

Drinks_bar -

```

53      6: 49: int server_fd = socket(AF_INET, SOCK_STREAM, 0);
54      6: 50: if (server_fd < 0)
55      -: 51: {
56      #####: 52:     perror("TCP socket failed");
57      #####: 53:     exit(EXIT_FAILURE);
58      -: 54: }
59      -: 55:

```

```

73      3: 69: if (listen(server_fd, 5) < 0)
74      -: 70: {
75      #####: 71:     perror("TCP listen failed");
76      #####: 72:     close(server_fd);
77      #####: 73:     exit(EXIT_FAILURE);
78      -: 74: }
79      -: 75:
80      3: 76: int udp_fd = socket(AF_INET, SOCK_DGRAM, 0);
81      3: 77: if (udp_fd < 0)
82      -: 78: {
83      #####: 79:     perror("UDP socket failed");
84      #####: 80:     close(server_fd);
85      #####: 81:     exit(EXIT_FAILURE);
86      -: 82: }

```

```

94      3: 90: if (bind(udp_fd, (struct sockaddr *)&udp_addr, sizeof(udp_addr)) < 0)
95      -: 91: {
96      #####: 92:     perror("UDP bind failed");
97      #####: 93:     close(server_fd);
98      #####: 94:     close(udp_fd);
99      #####: 95:     exit(EXIT_FAILURE);
00      -: 96: }

```

```

-: 125: {
#####: 126:     perror("select error");
#####: 127:     break;
-: 128: }

```

```

#####: 137:     perror("accept failed");
#####: 138:     continue;
-: 139: }

```

molecule_requester -

```

25      #####: 21:     perror("socket failed");
26      #####: 22:     exit(EXIT_FAILURE);
27      -: 23: }
28      -: 24:
29      -: 25:     struct sockaddr_in server_addr;
30      9: 26:     memset(&server_addr, 0, sizeof(server_addr));
31      9: 27:     server_addr.sin_family = AF_INET;
32      9: 28:     server_addr.sin_port = htons(port);
33      9: 29:     if (inet_pton(AF_INET, server_ip, &server_addr.sin_addr) <= 0) {
34      #####: 30:         perror("invalid address");
35      #####: 31:         exit(EXIT_FAILURE);
36      -: 32: }

```

```

#####: 50:     perror("sendto failed");
#####: 51:     continue;
-: 52: }
-: 53:
-: 54:
21: 55:     socklen_t addrlen = sizeof(server_addr);
21: 56:     int bytes_received = recvfrom(sockfd, response, sizeof(response) - 1, 0,
-: 57:                                     (struct sockaddr *)&server_addr, &addrlen);
21*: 58:     if (bytes_received < 0) {
#####: 59:         perror("recvfrom failed");
#####: 60:         continue;
-: 61: }

```

atom_supplier -

```

#####: 31:     fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(status));
#####: 32:     return 1;
-: 33: }
-: 34:
7: 35:     int sock = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
7: 36:     if (sock < 0)
-: 37:     {
#####: 38:         perror("socket failed");
#####: 39:         freeaddrinfo(res);
#####: 40:         return 1;
-: 41:     }
-: 42:
7: 43:     if (connect(sock, res->ai_addr, res->ai_addrlen) < 0)
-: 44:     {
#####: 45:         perror("Connection failed");
#####: 46:         freeaddrinfo(res);
#####: 47:         close(sock);
#####: 48:         return 1;
-: 49:     }

```

```

#####: 94:         perror("Send failed");
#####: 95:         break;
-: 96:     }
-: 97:
8: 98:     unsigned int response = 0;
8: 99:     ssize_t bytes_received = recv(sock, &response, sizeof(response), 0);
8: 100:     if (bytes_received != sizeof(response))
-: 101:     {
#####: 102:         fprintf(stderr, "Failed to receive proper response from server\n");
#####: 103:         break;
-: 104:     }

```

```

####: 113:         perror("Send failed");
####: 114:         break;
-: 115:     }
-: 116:
2: 117:     unsigned int response = 0;
2: 118:     ssize_t bytes_received = recv(sock, &response, sizeof(response), 0);
2: 119:     if (bytes_received != sizeof(response))
-: 120:     {
####: 121:         fprintf(stderr, "Failed to receive proper response from server\n");
####: 122:         break;
-: 123:     }

```

שלב רביעי:

בשלב זה נדרשנו להוסיף דגלים לשורת ההרצה שחלקם הכרחיים וחלקם לא .

-o / --oxygen	אופציונלי	מספר האטומים של חמצן
-c / --carbon	אופציונלי	מספר האטומים של פחמן
-h / --hydrogen	אופציונלי	מספר האטומים של מימן
-t / --timeout	אופציונלי	בשניות TIMEOUT
-T / --tcp-port	הכרחי	פורט TCP

-U / --udp-port	הכרחי	פורט UDP
-----------------	-------	----------

כמו כן אין סדר לקבלת הארגומנטים

לדוגמא:

./drinks_bar -t 60 -T 5555 -o 12 -U 7777

./drinks_bar -t 60 -T 5555 -U 7777 -o 12

הוספנו בקוד של השרת את הקוד הבא:

```

44 int tcp_port = -1;
45 int udp_port = -1;
46
47 static struct option long_options[] = {
48     {"oxygen", required_argument, 0, 'o'},
49     {"carbon", required_argument, 0, 'c'},
50     {"hydrogen", required_argument, 0, 'h'},
51     {"timeout", required_argument, 0, 't'},
52     {"tcp-port", required_argument, 0, 'T'},
53     {"udp-port", required_argument, 0, 'U'},
54     {0, 0, 0, 0}};
55
56 int opt;
57 while ((opt = getopt_long(argc, argv, "o:c:h:t:T:U:", long_options, NULL)) != -1)
58 {
59     switch (opt)
60     {
61     case 'o':
62         oxygen = strtoull(optarg, NULL, 10);
63         break;
64     case 'c':
65         carbon = strtoull(optarg, NULL, 10);
66         break;
67     case 'h':
68         hydrogen = strtoull(optarg, NULL, 10);
69         break;
70     case 't':
71         timeout = atoi(optarg);
72         break;
73     case 'T':
74         tcp_port = atoi(optarg);
75         break;
76     case 'U':
77         udp_port = atoi(optarg);
78         break;
79     default:
80         fprintf(stderr, "Usage: %s -T <tcp_port> -U <udp_port> [--oxygen N | -o N] [--carbon N | -c N] [--hydrogen N | -h N] [--timeout N | -t N]\n", argv[0]);
81         exit(EXIT_FAILURE);
82     }
83 }

```

וכאן נדאג לפרמטרים ההכרחיים

```

if (tcp_port < 0 || udp_port < 0)
{
    fprintf(stderr, "Error: --tcp-port (-T) and --udp-port (-U) are required.\n");
    exit(EXIT_FAILURE);
}

```

בנוסף הלקוחות יקבלו ארגומנטים בעזרת אופציות לדוגמא:
./atom_supplier -h <hostname> -p <port>

שינוי בקבצים atom_warehouse ובmolecule_requester

```

18 while ((opt = getopt(argc, argv, "h:p:")) != -1)
19 {
20     switch (opt)
21     {
22         case 'h':
23             host = optarg;
24             break;
25         case 'p':
26             port_str = optarg;
27             break;
28         default:
29             fprintf(stderr, "Usage: %s -h <host> -p <port>\n", argv[0]);
30             return 1;
31     }
32 }
33
34 if (!host || !port_str)
35 {
36     fprintf(stderr, "Error: -h <host> and -p <port> are required.\n");
37     return 1;
38 }
39

```

דוגמת הרצה:

```

● ron@Ron:~/OS_EX2/EX4$ ./drinks_bar -t 60 -T 4444 -o 12 -U 5555
Starting with H=0, O=12, C=0, timeout=60
Server is listening on TCP 4444 and UDP 5555
New client connected: 127.0.0.1
Atoms warehouse: H=0, O=12, C=100
Timeout of 60 seconds reached. Server shutting down.
Server shut down.....

```

```

● ron@Ron:~/OS_EX2/EX4$ ./atom_supplier -h 127.0.0.1 -p 4444
Connected to server at 127.0.0.1:4444
Enter command ADD CARBON , OXYGEN , HYDROGEN: ADD CARBON 100
Server returned count: CARBON = 100
Enter command ADD CARBON , OXYGEN , HYDROGEN: 

```

יכולנו להריץ גם כך

```

● ron@Ron:~/OS_EX2/EX4$ ./drinks_bar -t 60 -T 1111 -U 7777 -o 44
Starting with H=0, O=44, C=0, timeout=60
Server is listening on TCP 1111 and UDP 7777
Timeout of 60 seconds reached. Server shutting down.
Server shut down.....

```

Code coverage -

הרצנו סקריפט כמו בשלבים הקודמים

```
Generating coverage reports...
File 'drinks_bar.c'
Lines executed:92.83% of 237
Creating 'drinks_bar.c.gcov'

Lines executed:92.83% of 237
File 'atom_supplier.c'
Lines executed:75.32% of 77
Creating 'atom_supplier.c.gcov'

Lines executed:75.32% of 77
File 'molecule_requester.c'
Lines executed:82.22% of 45
Creating 'molecule_requester.c.gcov'

Lines executed:82.22% of 45
```

Drink_bar -

```
7: 93: int server_fd = socket(AF_INET, SOCK_STREAM, 0);
7: 94: if (server_fd < 0)
-: 95: {
#####: 96:     perror("TCP socket failed");
#####: 97:     exit(EXIT_FAILURE);
-: 98: }
```

```
119 #####: 115:     perror("TCP listen failed");
120 #####: 116:     close(server_fd);
121 #####: 117:     exit(EXIT_FAILURE);
122 -: 118: }
123 -: 119:
124 4: 120: int udp_fd = socket(AF_INET, SOCK_DGRAM, 0);
125 4: 121: if (udp_fd < 0)
126 -: 122: {
127 #####: 123:     perror("UDP socket failed");
128 #####: 124:     close(server_fd);
129 #####: 125:     exit(EXIT_FAILURE);
130 -: 126: }
131 -: 127:
132 -: 128: struct sockaddr_in udp_addr;
133 4: 129: memset(&udp_addr, 0, sizeof(udp_addr));
134 4: 130: udp_addr.sin_family = AF_INET;
135 4: 131: udp_addr.sin_addr.s_addr = INADDR_ANY;
136 4: 132: udp_addr.sin_port = htons(udp_port);
137 -: 133:
138 4: 134: if (bind(udp_fd, (struct sockaddr *)&udp_addr, sizeof(udp_addr)) < 0)
139 -: 135: {
140 #####: 136:     perror("UDP bind failed");
141 #####: 137:     close(server_fd);
142 #####: 138:     close(udp_fd);
143 #####: 139:     exit(EXIT_FAILURE);
144 -: 140: }
145 -: 141:
```



```

#####: 186:         perror("select error");
#####: 187:         break;
-: 188:     }
-: 189:
695024: 190:     if (activity > 0)
-: 191:     {
695024: 192:         last_activity = time(NULL);
-: 193:     }
-: 194:
695024: 195:     if (FD_ISSET(server_fd, &readfds))
-: 196:     {
-: 197:         struct sockaddr_in client_addr;
6: 198:         socklen_t client_len = sizeof(client_addr);
6: 199:         int new_socket = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
6*: 200:         if (new_socket < 0)
-: 201:         {
#####: 202:             perror("accept failed");
#####: 203:             continue;
-: 204:         }

```

Molecule_requester -

```

#####: 37:         perror("socket failed");
#####: 38:         exit(EXIT_FAILURE);
-: 39:     }
-: 40:
-: 41:     struct sockaddr_in server_addr;
9: 42:     memset(&server_addr, 0, sizeof(server_addr));
9: 43:     server_addr.sin_family = AF_INET;
9: 44:     server_addr.sin_port = htons(port);
9: 45:     if (inet_pton(AF_INET, server_ip, &server_addr.sin_addr) <= 0) {
#####: 46:         perror("invalid address");
#####: 47:         exit(EXIT_FAILURE);
-: 48:     }

```

```

#####: 65:         perror("sendto failed");
#####: 66:         continue;
-: 67:     }
-: 68:
21: 69:     socklen_t addrlen = sizeof(server_addr);
21: 70:     int bytes_received = recvfrom(sockfd, response, sizeof(response) - 1, 0,
-: 71:                                   (struct sockaddr *)&server_addr, &addrlen);
21*: 72:     if (bytes_received < 0) {
#####: 73:         perror("recvfrom failed");
#####: 74:         continue;
-: 75:     }
-: 76:

```

Atom_supplier -

```

#####: 48:         fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(status));
#####: 49:         return 1;
-: 50:     }
-: 51:
7: 52:     int sock = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
7: 53:     if (sock < 0)
-: 54:     {
#####: 55:         perror("socket failed");
#####: 56:         freeaddrinfo(res);
#####: 57:         return 1;
-: 58:     }
-: 59:
7: 60:     if (connect(sock, res->ai_addr, res->ai_addrlen) < 0)
-: 61:     {
#####: 62:         perror("Connection failed");
#####: 63:         freeaddrinfo(res);
#####: 64:         close(sock);
#####: 65:         return 1;
-: 66:     }

```

```

#####: 110:         perror("Send failed");
#####: 111:         break;
-: 112:     }
-: 113:
8: 114:     unsigned int response = 0;
8: 115:     ssize_t bytes_received = recv(sock, &response, sizeof(response), 0);
8: 116:     if (bytes_received != sizeof(response))
-: 117:     {
#####: 118:         fprintf(stderr, "Failed to receive proper response from server\n");
#####: 119:         break;
-: 120:     }
-: 121:
8: 122:     printf("Server returned count: %s = %u\n", atom_type, response);
-: 123: }
-: 124: else
-: 125: {
2: 126:     ssize_t bytes_sent = send(sock, buffer, strlen(buffer), 0);
2: 127:     if (bytes_sent < 0)
-: 128:     {
#####: 129:         perror("Send failed");
#####: 130:         break;
-: 131:     }

```

שלב 5:

נדרשנו לשדרג את השרת כך שיתמוך בתקשורת על פני UDS מסוגי stream, datagram לאותם תפקידים כמו TCP וUDP בהתאם.

זה סוקט שמתקשר באותו מחשב בין תהליכים.

בפונקציית ה socket נגדיר AF_UNIX.

כאן אנו מזדהים לפי מחרוזת אחת, והיא מייצגת מסלול לקובץ

לשרת הוספנו את האופציות הבאות

-s/--stream-path<UDS stream file path>

-d/--datagram-path<UDS datagram filepath>

ואת הלקוחות שדרגו כך שיכולו לקבל במקום כתובת ופורט ארגומנט -f <UDS socket file path>

בשלב זה נדון על סוקטים שמתקשרים בתוך המחשב בין תהליכים בניגוד לסוקטים שראינו קודם.

בצד השרת drink_bar

- הוספנו תמיכה בUDS הוספנו שני סוגים (SOCK_STREAM-TCP) ו(SOCK_DGRAM-UDP)

```
54 char *uds_stream_path = NULL;
55 char *uds_datagram_path = NULL;
56 char *uds_socket_path = NULL;
```

- הוספנו דגלים חדשים על מנת שהשרת יתמוך בהם

```
65 {"stream-path", required_argument, 0, 's'},
66 {"datagram-path", required_argument, 0, 'd'},
67 {"socket-path", required_argument, 0, 'f'}, // For backward compatibility
68 {0, 0, 0, 0};
69
```

- פתחנו סוקט UDS מסוג Stream, וגרמנו לקיום קשר דרך UDS

```
uds_stream_fd = socket(AF_UNIX, SOCK_STREAM, 0);
if (uds_stream_fd < 0)
{
    perror("UDS stream socket failed");
    cleanup_uds_files(NULL, NULL, NULL);
    if (server_fd >= 0) close(server_fd);
    if (udp_fd >= 0) close(udp_fd);
    exit(EXIT_FAILURE);
}

struct sockaddr_un stream_addr;
memset(&stream_addr, 0, sizeof(stream_addr));
stream_addr.sun_family = AF_UNIX;
strncpy(stream_addr.sun_path, stream_path, sizeof(stream_addr.sun_path) - 1);

unlink(stream_path); // Remove existing socket file
if (bind(uds_stream_fd, (struct sockaddr *)&stream_addr, sizeof(stream_addr)) < 0)
{
    perror("UDS stream bind failed");
    cleanup_uds_files(stream_path, NULL, NULL);
    if (server_fd >= 0) close(server_fd);
    if (udp_fd >= 0) close(udp_fd);
    close(uds_stream_fd);
    exit(EXIT_FAILURE);
}
```

בצד הלקוחות

molecule_requester.c-

בקובץ הלקוח המעודכן נוספה תמיכה מלאה בעבודה עם סוקט מסוג Unix Domain Socket (UDS) לצד UDP המשתמש יכול לבחור בין שימוש בפרוטוקול UDP (עבור תקשורת דרך הרשת) לבין UDS (עבור תקשורת מקומית בין תהליכים), באמצעות דגלים בשורת הפקודה <IP> -h או <port> -p עבור UDP או <path> -f עבור UDS. כאשר נעשה שימוש ב-UDS הלקוח מחויב לבצע bind() לנתיב סוקט זמני כדי שהשרת יוכל לשלוח אליו תגובות. לאחר סיום הריצה, הקובץ הזמני נמחק אוטומטית בעזרת unlink().

תקשורת דו-כיוונית תקינה בין הלקוח לשרת גם בפרוטוקול UDS מסוג Datagram הקוד משתמש במשתנה use_uds כדי להבחין בין שני המצבים ומפצל בהתאם את הלוגיקה של יצירת הסוקט, שליחה (sendto) וקבלה (recvfrom).

atom_supplier.c-

בקובץ לקוח זה נוספה תמיכה בבחירה בין התחברות לשרת דרך TCP באמצעות כתובת IP ופורט לבין התחברות לשרת מקומי דרך UDS מסוג SOCK_STREAM הבחירה מתבצעת דרך הדגלים -h או -p עבור TCP או -s עבור UDS

הקוד מבצע התחברות לשרת המתאים: עבור TCP באמצעות getaddrinfo() ו-connect() לכתובת רשת, ועבור UDS באמצעות יצירת כתובת מקומית (sockaddr_un) וחיבור לקובץ סוקט. לאחר ההתחברות, הלקוח שולח פקודות כגון ADD, EXIT, SHUTDOWN, ומקבל תגובה מהשרת בהתאם. הפקודות נשלחות כ־text דרך send(), והתשובות מתקבלות או כ־טקסט (במקרה של שגיאה), או כ־מספר שלם (unsigned int) עבור הצלחת ADD.

- דוגמת הרצה
בצד השרת

```
ron@Ron:~/OS_EX2/EX5$ ./drinks_bar --stream-path /tmp/server_stream -d /tmp/server_datagram
Starting with H=0, O=0, C=0, timeout=-1
UDS stream socket listening on /tmp/server_stream
UDS datagram socket listening on /tmp/server_datagram
```

בצד הלקוח

```
ron@Ron:~/OS_EX2/EX5$ ./atom_supplier -s /tmp/server_stream
Connected to UDS server at /tmp/server_stream
Connection established. You can now send commands.
Available commands:
  ADD HYDROGEN <amount> - Add hydrogen atoms
  ADD OXYGEN <amount>    - Add oxygen atoms
  ADD CARBON <amount>   - Add carbon atoms
  EXIT                   - Disconnect from server
  SHUTDOWN               - Shutdown server

Enter command: ADD CARBON 4
Server returned count: CARBON = 4
Enter command: ADD HYDROGEN 10
Server returned count: HYDROGEN = 10
Enter command: ADD OXYGEN 10
Server returned count: OXYGEN = 10
Enter command: ADD CARBON 4
```

בצד הלקוח:

```
ron@Ron:~/OS_EX2/EX5$ ./molecule_requester -f /tmp/server_datagram
Connected to UDS server at /tmp/server_datagram
Enter DELIVER commands (e.g., DELIVER WATER 2), or type EXIT to quit:
> DELIVER WATER 1
Server response: CANNOT_DELIVER
> DELIVER WATER 1
Server response: DELIVERED
> 
```

Code coverage -

```
Generating coverage reports...
File 'drinks_bar.c'
Lines executed:86.24% of 378
Creating 'drinks_bar.c.gcov'

Lines executed:86.24% of 378
File 'atom_supplier.c'
Lines executed:73.83% of 149
Creating 'atom_supplier.c.gcov'

Lines executed:73.83% of 149
File 'molecule_requester.c'
Lines executed:77.53% of 89
Creating 'molecule_requester.c.gcov'

Lines executed:77.53% of 89

Cleaning up temporary files...
Complete coverage test finished!
```

ננתח את התוצאות ונסביר מדוע הן בסדר

Drinks_bar -

```
115 #####: 111:      fprintf(stderr, "Error: At least one socket type must be configured (TCP, UDP, or UDS).\n");
116 #####: 112:      exit(EXIT_FAILURE);
117 -: 113:  }
118 -: 114:
119 8: 115:  printf("Starting with H=%llu, O=%llu, C=%llu, timeout=%d\n", hydrogen, oxygen, carbon, timeout);
120 -: 116:
121 -: 117:  // Socket file descriptors
122 8: 118:  int server_fd = -1, udp_fd = -1, uds_stream_fd = -1, uds_datagram_fd = -1;
123 -: 119:
124 -: 120:  // Setup TCP socket if requested
125 8: 121:  if (tcp_port >= 0)
126 -: 122:  {
127 6: 123:      server_fd = socket(AF_INET, SOCK_STREAM, 0);
128 6: 124:      if (server_fd < 0)
129 -: 125:      {
130 #####: 126:          perror("TCP socket failed");
131 #####: 127:          exit(EXIT_FAILURE);
132 -: 128:      }
133 -: 129:
134 -: 130:  struct sockaddr_in address;
135 6: 131:  memset(&address, 0, sizeof(address));
136 6: 132:  address.sin_family = AF_INET;
137 6: 133:  address.sin_addr.s_addr = INADDR_ANY;
138 6: 134:  address.sin_port = htons(tcp_port);
139 -: 135:
140 6: 136:  if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0)
141 -: 137:  {
142 2: 138:      perror("TCP bind failed");
143 2: 139:      close(server_fd);
144 2: 140:      exit(EXIT_FAILURE);
145 -: 141:  }
146 -: 142:
147 4: 143:  if (listen(server_fd, 5) < 0)
148 -: 144:  {
149 #####: 145:      perror("TCP listen failed");
150 #####: 146:      close(server_fd);
151 #####: 147:      exit(EXIT_FAILURE);
152 -: 148:  }
153 4: 149:  printf("TCP server listening on port %d\n", tcp_port);
154 -: 150: }
```

```

162 #####: 158:         perror("UDP socket failed");
163 #####: 159:         if (server_fd >= 0) close(server_fd);
164 #####: 160:         exit(EXIT_FAILURE);
165 -: 161:     }
166 -: 162:
167 -: 163:     struct sockaddr_in udp_addr;
168 4: 164:     memset(&udp_addr, 0, sizeof(udp_addr));
169 4: 165:     udp_addr.sin_family = AF_INET;
170 4: 166:     udp_addr.sin_addr.s_addr = INADDR_ANY;
171 4: 167:     udp_addr.sin_port = htons(udp_port);
172 -: 168:
173 4: 169:     if (bind(udp_fd, (struct sockaddr *)&udp_addr, sizeof(udp_addr)) < 0)
174 -: 170:     {
175 #####: 171:         perror("UDP bind failed");
176 #####: 172:         if (server_fd >= 0) close(server_fd);
177 #####: 173:         close(udp_fd);
178 #####: 174:         exit(EXIT_FAILURE);
179 -: 175:     }
180 4: 176:     printf("UDP server listening on port %d\n", udp_port);
181 -: 177: }
182 -: 178:
183 -: 179: // Setup UDS stream socket if requested
184 6: 180: if (uds_stream_path || uds_socket_path)
185 -: 181: {
186 2*: 182:     const char *stream_path = uds_stream_path ? uds_stream_path : uds_socket_path;
187 -: 183:
188 2: 184:     uds_stream_fd = socket(AF_UNIX, SOCK_STREAM, 0);
189 2: 185:     if (uds_stream_fd < 0)
190 -: 186:     {
191 #####: 187:         perror("UDS stream socket failed");
192 #####: 188:         cleanup_uds_files(NULL, NULL, NULL);
193 #####: 189:         if (server_fd >= 0) close(server_fd);
194 #####: 190:         if (udp_fd >= 0) close(udp_fd);
195 #####: 191:         exit(EXIT_FAILURE);
196 -: 192:     }

```

```

#####: 202:         perror("UDS stream bind failed");
#####: 203:         cleanup_uds_files(stream_path, NULL, NULL);
#####: 204:         if (server_fd >= 0) close(server_fd);
#####: 205:         if (udp_fd >= 0) close(udp_fd);
#####: 206:         close(uds_stream_fd);
#####: 207:         exit(EXIT_FAILURE);
-: 208:     }
-: 209:
2: 210:     if (listen(uds_stream_fd, 5) < 0)
-: 211:     {
#####: 212:         perror("UDS stream listen failed");
#####: 213:         cleanup_uds_files(stream_path, NULL, NULL);
#####: 214:         if (server_fd >= 0) close(server_fd);
#####: 215:         if (udp_fd >= 0) close(udp_fd);
#####: 216:         close(uds_stream_fd);
#####: 217:         exit(EXIT_FAILURE);
-: 218:     }
-: 219:     printf("UDS stream socket listening on %s\n", stream_path);
-: 220: }
-: 221:
-: 222: // Setup UDS datagram socket if requested
6: 223: if (uds_datagram_path)
-: 224: {
3: 225:     uds_datagram_fd = socket(AF_UNIX, SOCK_DGRAM, 0);
3: 226:     if (uds_datagram_fd < 0)
-: 227:     {
#####: 228:         perror("UDS datagram socket failed");
#####: 229:         cleanup_uds_files(uds_stream_path, NULL, uds_socket_path);
#####: 230:         if (server_fd >= 0) close(server_fd);
#####: 231:         if (udp_fd >= 0) close(udp_fd);
#####: 232:         if (uds_stream_fd >= 0) close(uds_stream_fd);
#####: 233:         exit(EXIT_FAILURE);
-: 234:     }
-: 235:

```

```

#####: 244:         perror("UDS datagram bind failed");
#####: 245:         cleanup_uds_files(uds_stream_path, uds_datagram_path, uds_socket_path);
#####: 246:         if (server_fd >= 0) close(server_fd);
#####: 247:         if (udp_fd >= 0) close(udp_fd);
#####: 248:         if (uds_stream_fd >= 0) close(uds_stream_fd);
#####: 249:         close(uds_datagram_fd);
#####: 250:         exit(EXIT_FAILURE);
-: 251:     }

```

```

#####: 322:         perror("TCP accept failed");
-: 323:     }

```

```

#####: 346:         perror("UDS stream accept failed");
-: 347:     }

```

```

2: 655:         target = &carbon;
#####: 656:     else {
#####: 657:         const char *error_msg = "ERROR: Unknown atom type.\n";
#####: 658:         send(client_fd, error_msg, strlen(error_msg), 0);
#####: 659:         continue;}
-: 660:

```

Atom_supplier -

```

#####: 35:         fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(status));
#####: 36:         return -1;
-: 37:     }
-: 38:
8: 39:     int sock = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
8: 40:     if (sock < 0)
-: 41:     {
#####: 42:         perror("TCP socket failed");
#####: 43:         freeaddrinfo(res);
#####: 44:         return -1;
-: 45:     }
-: 46:
8: 47:     if (connect(sock, res->ai_addr, res->ai_addrlen) < 0)
-: 48:     {
#####: 49:         perror("TCP connection failed");
#####: 50:         freeaddrinfo(res);
#####: 51:         close(sock);
#####: 52:         return -1;
-: 53:     }
-: 54:
8: 55:     printf("Connected to TCP server at %s:%s\n", host, port_str);
8: 56:     freeaddrinfo(res);
8: 57:     return sock;
-: 58: }
-: 59:
14: 60: int connect_uds(const char *socket_path)
-: 61: {
14: 62:     int sock = socket(AF_UNIX, SOCK_STREAM, 0);
14: 63:     if (sock < 0)
-: 64:     {
#####: 65:         perror("UDS socket failed");
#####: 66:         return -1;
-: 67:     }
-: 68:
-: 69:     struct sockaddr_un server_addr;
14: 70:     memset(&server_addr, 0, sizeof(server_addr));
14: 71:     server_addr.sun_family = AF_UNIX;
-: 72:
14: 73:     if (strlen(socket_path) >= sizeof(server_addr.sun_path))
-: 74:     {
#####: 75:         fprintf(stderr, "Socket path too long: %s\n", socket_path);
#####: 76:         close(sock);
#####: 77:         return -1;

```

```

#####: 84:         perror("UDS connection failed");
#####: 85:         close(sock);
#####: 86:         return -1;
-: 87:     }

```

```

#####: 134:         fprintf(stderr, "Error: Must specify either TCP (-h and -p) or UDS (-f) connection.\n\n");
#####: 135:         print_usage(argv[0]);
#####: 136:         return 1;
-: 137:     }
-: 138:
22: 139:     if (tcp_mode && uds_mode)
-: 140:     {
#####: 141:         fprintf(stderr, "Error: Cannot specify both TCP and UDS connections simultaneously.\n\n");
#####: 142:         print_usage(argv[0]);
#####: 143:         return 1;
-: 144:     }
-: 145:
22: 146:     if (tcp_mode && (!host || !port_str))
-: 147:     {
#####: 148:         fprintf(stderr, "Error: For TCP connection, both -h <host> and -p <port> are required.\n\n");
#####: 149:         print_usage(argv[0]);
#####: 150:         return 1;

```

```

#####: 166:         fprintf(stderr, "Failed to establish connection.\n");
#####: 167:         return 1;
-: 168:     }

```

```

##### 233:         perror("Send failed");
##### 234:         break;
-: 235:     }
-: 236:
-: 237:     // Expect a binary response (unsigned int) for valid ADD commands
30: 238:     unsigned int response = 0;
30: 239:     ssize_t bytes_received = recv(sock, &response, sizeof(response), 0);
30: 240:     if (bytes_received != sizeof(response))
-: 241:     {
##### 242:         fprintf(stderr, "Failed to receive proper response from server\n");
-: 243:         break;
-: 244:     }
-: 245:
30: 246:     printf("Server returned count: %s = %u\n", atom_type, response);
-: 247: }
-: 248: else
-: 249: {
-: 250:     // Send invalid command and expect text error response
2: 251:     ssize_t bytes_sent = send(sock, buffer, strlen(buffer), 0);
2: 252:     if (bytes_sent < 0)
-: 253:     {
##### 254:         perror("Send failed");
##### 255:         break;
-: 256:     }
-: 257:
2: 258:     char error_response[BUFFER_SIZE] = {0};
2: 259:     ssize_t bytes_received = recv(sock, error_response, sizeof(error_response) - 1, 0);
2: 260:     if (bytes_received > 0)
-: 261:     {
2: 262:         error_response[bytes_received] = '\0';
2: 263:         printf("Server response: %s", error_response);
-: 264:     }
##### 265:     else if (bytes_received == 0)
-: 266:     {
##### 267:         printf("Server closed connection.\n");
##### 268:         break;
-: 269:     }
-: 270:     else
-: 271:     {
##### 272:         perror("Failed to receive response from server");
##### 273:         break;
-: 274:     }

```

molecule_requester -

```

54 ##### 50:         fprintf(stderr, "Error: -f <UDS_socket_path> is required for UDS mode.\n");
55 ##### 51:         exit(EXIT_FAILURE);
56 -: 52:     }
57 -: 53: } else {
58 4: 54:     if (!server_ip || port <= 0) {
59 ##### 55:         fprintf(stderr, "Error: both -h <server_ip> and -p <port> are required for UDP mode.\n");
60 ##### 56:         exit(EXIT_FAILURE);
61 -: 57:     }
62 -: 58: }
63 -: 59:
64 -: 60: int sockfd;
65 -: 61: char input[BUFFER_SIZE];
66 -: 62: char response[BUFFER_SIZE];
67 -: 63:
68 9: 64: if (use_uds) {
69 5: 65:     sockfd = socket(AF_UNIX, SOCK_DGRAM, 0);
70 5: 66:     if (sockfd < 0) {
71 ##### 67:         perror("UDS socket failed");
72 ##### 68:         exit(EXIT_FAILURE);
73 -: 69:     }
74 -: 70:
75 -: 71:     // קביעת כתובת זמנית ללקוח
76 -: 72:     struct sockaddr_un client_addr;
77 5: 73:     memset(&client_addr, 0, sizeof(client_addr));
78 5: 74:     client_addr.sun_family = AF_UNIX;
79 5: 75:     snprintf(client_addr.sun_path, sizeof(client_addr.sun_path), "/tmp/client_%d", getpid());
80 5: 76:     unlink(client_addr.sun_path); // מניעת התנגשות
81 -: 77:
82 5: 78:     if (bind(sockfd, (struct sockaddr *)&client_addr, sizeof(client_addr)) < 0) {
83 ##### 79:         perror("bind failed");
84 ##### 80:         exit(EXIT_FAILURE);
85 -: 81:     }

```



```
##### 102:         perror("sendto failed");
##### 103:         continue;
-: 104:     }
-: 105:
23: 106:         socklen_t addrlen = sizeof(server_addr);
23: 107:         int bytes_received = recvfrom(sockfd, response, sizeof(response) - 1,
-: 108:                                     (struct sockaddr *)&server_addr, &addrlen);
23*: 109:         if (bytes_received < 0) {
##### 110:             perror("recvfrom failed");
##### 111:             continue;
-: 112:         }
-: 113:
23: 114:         response[bytes_received] = '\0';
23: 115:         printf("Server response: %s\n", response);
-: 116:     }
-: 117:
5: 118:         unlink(client_addr.sun_path); // ניקוי בסיס
-: 119:
-: 120:     } else {
-: 121:         // לא UDS רגיל חשב
4: 122:         sockfd = socket(AF_INET, SOCK_DGRAM, 0);
4: 123:         if (sockfd < 0) {
##### 124:             perror("socket failed");
##### 125:             exit(EXIT_FAILURE);
-: 126:         }
-: 127:
-: 128:         struct sockaddr_in server_addr;
4: 129:         memset(&server_addr, 0, sizeof(server_addr));
4: 130:         server_addr.sin_family = AF_INET;
4: 131:         server_addr.sin_port = htons(port);
4: 132:         if (inet_pton(AF_INET, server_ip, &server_addr.sin_addr) <= 0) {
##### 133:             perror("invalid address");
##### 134:             exit(EXIT_FAILURE);
-: 135:         }

```

```
##### 150:         perror("sendto failed");
##### 151:         continue;
-: 152:     }
-: 153:
9: 154:         socklen_t addrlen = sizeof(server_addr);
9: 155:         int bytes_received = recvfrom(sockfd, response, sizeof(response) - 1, 0,
-: 156:                                     (struct sockaddr *)&server_addr, &addrlen);
9*: 157:         if (bytes_received < 0) {
##### 158:             perror("recvfrom failed");
##### 159:             continue;

```

אפשר לראות שכל מה שלא כוסה זה בעצם בדיקות .

שלב 6:

בשלב זה נרצה שהשרת יוכל לטעון את המלאי מקובץ, וגם בכל רגע נתון לשמור את המלאי הנוכחי באותו קובץ

השרת יקבל את הארגומנט f או -save-file ונתיב כלשהוא לקובץ

- אם הקובץ קיים אז הוא טוען ממנו את המלאי ומתעלם מאופציות האתחול של שלב 4 ובכל פעם שיש שינוי הוא הצטרך לעדכן בקובץ את המלאי
- אם הקובץ לא קיים ניצור את הקובץ עם מלאי התחלתי
- נשים לב שמספר תהליכים יכולים להריץ את השרת בו זמנית ויש לוודא שהם לא מפריעים אחד לשני בעזרת flock

דוגמת הרצה :

בצד השרת, נגדיר לו לטעון את המידע דרך קובץ בשם test.txt אבל הוא לא קיים ולכן השרת יגדיר את המלאי ההתחלתי ששמנו לו בשורת הפקודה ויצור את הקובץ.

```
ron@Ron:~/OS_EX2$ cd EX6
ron@Ron:~/OS_EX2/EX6$ ./drinks_bar --tcp-port 4444 --udp-port 5555 -o 12 -h 40 -c 44 --save-file test.txt
Save file test.txt not found, starting with default values
Starting with H=40, O=12, C=44, timeout=-1
Save file: test.txt
TCP server listening on port 4444
UDP server listening on port 5555
[]
```

בצד הלקוח נוסיף אטומים

```
ron@Ron:~/OS_EX2/EX6$ ./atom_supplier -h 127.0.0.1 -p 4456
Connected to TCP server at 127.0.0.1:4456
Connection established. You can now send commands.
Available commands:
  ADD HYDROGEN <amount> - Add hydrogen atoms
  ADD OXYGEN <amount>    - Add oxygen atoms
  ADD CARBON <amount>   - Add carbon atoms
  EXIT                   - Disconnect from server
  SHUTDOWN               - Shutdown server

Enter command: ADD CARBON 100
Server returned count: CARBON = 154
Enter command: ADD OXYGEN 100
Server returned count: OXYGEN = 121
Enter command: ADD HYDROGEN 100
Server returned count: HYDROGEN = 148
Enter command: []
```

נתסכל בקובץ ונראה :

```
EX6 > ≡ test.txt
1 148 121 154
```

ובשרת :

```
ron@Ron:~/OS_EX2/EX6$ ./drinks_bar --tcp-port 4456 --udp-port 6781 -o 12 -h 40 -c 44 --save-file test.txt
State loaded from test.txt: H=48, O=21, C=54
Starting with H=48, O=21, C=54, timeout=-1
Save file: test.txt
TCP server listening on port 4456
UDP server listening on port 6781
New TCP client connected: 127.0.0.1
Atoms warehouse: H=48, O=21, C=154
State saved to test.txt: H=48, O=21, C=154
Atoms warehouse: H=48, O=121, C=154
State saved to test.txt: H=48, O=121, C=154
Atoms warehouse: H=148, O=121, C=154
State saved to test.txt: H=148, O=121, C=154
```

רואים שבאמת יש שינויים מהמצב ההתחלתי.

ניגש ללקוח השני :

```
ron@Ron:~/OS_EX2/EX6$ ./molecule_requester -h 127.0.0.1 -p 6781
Enter DELIVER commands (e.g., DELIVER WATER 2), or type EXIT to quit:
> DELIVER WATER 1
Server response: DELIVERED
> DELIVER CARBON DIOXIDE 1
Server response: DELIVERED
>
```

בצד השרת:

```
UDP DELIVER: WATER (1 units)
Atoms warehouse after delivery: H=146, O=120, C=154
State saved to test.txt: H=146, O=120, C=154
UDP DELIVER: CARBON_DIOXIDE (1 units)
Atoms warehouse after delivery: H=146, O=118, C=153
State saved to test.txt: H=146, O=118, C=153
```

ונראה את השינויים בהפחתה בקובץ

```
EX6 > test.txt
1 146 118 153
```

- code coverage

```
Generating coverage reports...
File 'drinks_bar.c'
Lines executed:85.27% of 421
Creating 'drinks_bar.c.gcov'

Lines executed:85.27% of 421
File 'atom_supplier.c'
Lines executed:73.83% of 149
Creating 'atom_supplier.c.gcov'

Lines executed:73.83% of 149
File 'molecule_requester.c'
Lines executed:77.53% of 89
Creating 'molecule_requester.c.gcov'

Lines executed:77.53% of 89

Cleaning up temporary files...

Complete coverage test finished!
```

- Drinks_bar

```

35: #####: 51:         perror("Failed to open save file for writing");
36: #####: 52:         return -1;
37: -: 53:     }
38: -: 54:
39: 4: 55:     fprintf(file, "%llu %llu %llu\n", hydrogen, oxygen, carbon);
40: 4: 56:     fclose(file);
41: 4: 57:     printf("State saved to %s: H=%llu, O=%llu, C=%llu\n", filename, hydrogen, oxygen, carbon);
42: 4: 58:     return 0;
43: -: 59: }
44: -: 60:
45: -: 61: // Load server state from file
46: 2: 62: int load_state(unsigned long long *hydrogen, unsigned long long *oxygen, unsigned long long *carbon, const char
47: -: 63: {
48: 2: 64:     if (!filename) return 0;
49: -: 65:
50: 2: 66:     FILE *file = fopen(filename, "r");
51: 2: 67:     if (!file) {
52: 1: 68:         printf("Save file %s not found, starting with default values\n", filename);
53: 1: 69:         return 0; // Not an error, just no saved state
54: -: 70:     }
55: -: 71:
56: 1: 72:     if (fscanf(file, "%llu %llu %llu", hydrogen, oxygen, carbon) != 3) {
57: #####: 73:         fprintf(stderr, "Error reading save file %s\n", filename);
58: #####: 74:         fclose(file);
59: #####: 75:         return -1;
60: -: 76:     }

```

```

-: 154: {
#####: 155:     fprintf(stderr, "Error: At least one socket type must be configured (TCP, UDP, or UDS).\n");
#####: 156:     exit(EXIT_FAILURE);
-: 157: }
-: 158:
-: 159: // Try to load state from file if save file is specified
12: 160: int state_loaded = 0;
12: 161: if (save_file_path) {
2: 162:     int load_result = load_state(&hydrogen, &oxygen, &carbon, save_file_path);
2: 163:     if (load_result > 0) {
1: 164:         state_loaded = 1;
1: 165:     } else if (load_result < 0) {
#####: 166:         exit(EXIT_FAILURE);
-: 167:     }

```

```

#####: 184:         perror("TCP socket failed");
#####: 185:         exit(EXIT_FAILURE);
-: 186:     }
-: 187:
-: 188:     struct sockaddr_in address;
9: 189:     memset(&address, 0, sizeof(address));
9: 190:     address.sin_family = AF_INET;
9: 191:     address.sin_addr.s_addr = INADDR_ANY;
9: 192:     address.sin_port = htons(tcp_port);
-: 193:
9: 194:     if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0)
-: 195:     {
#####: 196:         perror("TCP bind failed");
#####: 197:         close(server_fd);
#####: 198:         exit(EXIT_FAILURE);
-: 199:     }
-: 200:
9: 201:     if (listen(server_fd, 5) < 0)
-: 202:     {
#####: 203:         perror("TCP listen failed");
#####: 204:         close(server_fd);
#####: 205:         exit(EXIT_FAILURE);
-: 206:     }

```

```

#####: 216:         perror("UDP socket failed");
#####: 217:         if (server_fd >= 0) close(server_fd);
#####: 218:         exit(EXIT_FAILURE);
-: 219:     }
-: 220:
-: 221:     struct sockaddr_in udp_addr;
9: 222:     memset(&udp_addr, 0, sizeof(udp_addr));
9: 223:     udp_addr.sin_family = AF_INET;
9: 224:     udp_addr.sin_addr.s_addr = INADDR_ANY;
9: 225:     udp_addr.sin_port = htons(udp_port);
-: 226:
9: 227:     if (bind(udp_fd, (struct sockaddr *)&udp_addr, sizeof(udp_addr)) < 0)
-: 228:     {
#####: 229:         perror("UDP bind failed");
#####: 230:         if (server_fd >= 0) close(server_fd);
#####: 231:         close(udp_fd);
#####: 232:         exit(EXIT_FAILURE);
-: 233:     }
-: 234:     printf("UDP server listening on port %d\n", udp_port);

```

```

#####: 245:         perror("UDS stream socket failed");
#####: 246:         cleanup_uds_files(NULL, NULL, NULL);
#####: 247:         if (server_fd >= 0) close(server_fd);
#####: 248:         if (udp_fd >= 0) close(udp_fd);
#####: 249:         exit(EXIT_FAILURE);
-: 250:     }
-: 251:
-: 252:     struct sockaddr_un stream_addr;
4: 253:     memset(&stream_addr, 0, sizeof(stream_addr));
4: 254:     stream_addr.sun_family = AF_UNIX;
4: 255:     strncpy(stream_addr.sun_path, stream_path, sizeof(stream_addr.sun_path) - 1);
-: 256:
4: 257:     unlink(stream_path); // Remove existing socket file
4: 258:     if (bind(uds_stream_fd, (struct sockaddr *)&stream_addr, sizeof(stream_addr)) < 0)
-: 259:     {
#####: 260:         perror("UDS stream bind failed");
#####: 261:         cleanup_uds_files(stream_path, NULL, NULL);
#####: 262:         if (server_fd >= 0) close(server_fd);
#####: 263:         if (udp_fd >= 0) close(udp_fd);
#####: 264:         close(uds_stream_fd);
#####: 265:         exit(EXIT_FAILURE);
-: 266:     }
-: 267:
4: 268:     if (listen(uds_stream_fd, 5) < 0)
-: 269:     {
#####: 270:         perror("UDS stream listen failed");
#####: 271:         cleanup_uds_files(stream_path, NULL, NULL);
#####: 272:         if (server_fd >= 0) close(server_fd);
#####: 273:         if (udp_fd >= 0) close(udp_fd);
#####: 274:         close(uds_stream_fd);
#####: 275:         exit(EXIT_FAILURE);
-: 276:     }

#####: 286:         perror("UDS datagram socket failed");
#####: 287:         cleanup_uds_files(uds_stream_path, NULL, uds_socket_path);
#####: 288:         if (server_fd >= 0) close(server_fd);
#####: 289:         if (udp_fd >= 0) close(udp_fd);
#####: 290:         if (uds_stream_fd >= 0) close(uds_stream_fd);
#####: 291:         exit(EXIT_FAILURE);
-: 292:     }
-: 293:
-: 294:     struct sockaddr_un datagram_addr;
3: 295:     memset(&datagram_addr, 0, sizeof(datagram_addr));
3: 296:     datagram_addr.sun_family = AF_UNIX;
3: 297:     strncpy(datagram_addr.sun_path, uds_datagram_path, sizeof(datagram_addr.sun_path) - 1);
-: 298:
3: 299:     unlink(uds_datagram_path); // Remove existing socket file
3: 300:     if (bind(uds_datagram_fd, (struct sockaddr *)&datagram_addr, sizeof(datagram_addr)) < 0)
-: 301:     {
#####: 302:         perror("UDS datagram bind failed");
#####: 303:         cleanup_uds_files(uds_stream_path, uds_datagram_path, uds_socket_path);
#####: 304:         if (server_fd >= 0) close(server_fd);
#####: 305:         if (udp_fd >= 0) close(udp_fd);
#####: 306:         if (uds_stream_fd >= 0) close(uds_stream_fd);
#####: 307:         close(uds_datagram_fd);
#####: 308:         exit(EXIT_FAILURE);
-: 309:     }

#####: 728:         const char *error_msg = "ERROR: Unknown atom type.\n";
#####: 729:         send(client_fd, error_msg, strlen(error_msg), 0);
#####: 730:         continue;
-: 731:     }

```

הקבצים של הלקוחות בדיוק כמו בשלב 5 כי לא שינו אותם .